A Survey of Optimizations for Large Language Model Inference

Bhargav Chirumamilla*
Independent Researcher
REPLACE_WITH_EMAIL

Abstract

Autoregressive large language models (LLMs) have made open-weight inference practical on everything from datacenter GPUs to laptop system-on-chips, but the same generation loop is simultaneously capacity-bound, bandwidth-bound, and latency-sensitive. This survey organizes the techniques that close that gap. We present a bottleneck-first taxonomy spanning ten families: weight and activation compression; key–value (KV) cache and context memory; attention and prefill kernels; speculative and parallel decoding; batching and request scheduling; model and expert parallelism; architecture and model-size choices; adapters; compilation and runtimes; and application-level controls. For each family we state the governing equations, the systems mapping onto hardware, and the primary papers. We then report a controlled empirical study of the local, single-stream subset of this taxonomy on Apple Silicon unified memory (Mac M3, 24 GB, and Mac M5 Max), using a public MLX harness that sweeps weight precision, KV quantization, prefill chunking, speculative decoding, context length, and two local runtimes. Stacking 4-bit weights with KV quantization and prefill tuning reduces Llama 3.1 8B peak memory from 16.3 GB to 5.1 GB and raises decode throughput from 5.6 to 19.9 tokens/s on the M3—about $3.6\times$—while speculative decoding yields up to $1.8\times$ further decode speedup when draft acceptance is high. We close with a decision procedure that maps user-visible failures (out-of-memory, slow first token, slow streaming, multi-tenant throughput) onto the right lever, and with open problems that remain after the current stack.

1 Introduction

Transformer language models [56] generate text one token at a time. That simple fact dominates systems design. Prefill—encoding the prompt—is highly parallel and often compute- or quadratic-attention bound. Decode—emitting the reply—is sequential, typically memory-bandwidth bound, and grows a KV cache linearly with sequence length [46, 58]. Production serving adds a third axis: many concurrent requests that must share scarce high-bandwidth memory [32, 61].

The last three years produced an unusually dense literature on *inference* optimizations, as distinct from training. Weight quantization [18, 37, 13] shrinks the static checkpoint. FlashAttention [12, 11] removes the $O(n^2)$ materialization of attention scores. PagedAttention and continuous batching [32, 61] make multi-user serving memory-efficient. Speculative decoding [34, 9] amortizes expensive target forwards. Disaggregated prefill/decode [44, 64, 2] splits the two phases across pools. Local runtimes such as MLX [5, 6] and llama.cpp [21] bring the same ideas to unified-memory laptops [4].

Prior surveys cover serving systems [40] and efficient inference more broadly [65]. This paper contributes three things they do not jointly provide: (i) a single bottleneck-first taxonomy that includes

*Corresponding author. Update affiliation and email before arXiv upload.

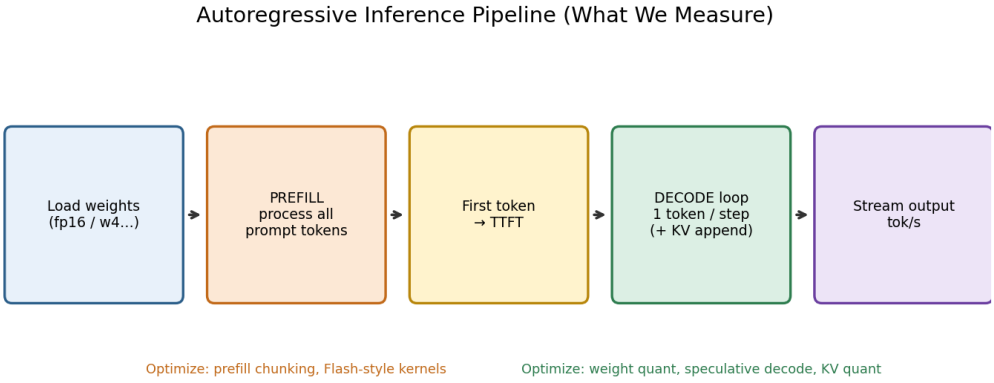

Figure 1: Single-request inference pipeline: load weights, prefill the prompt (TTFT), then autoregressive decode (tokens/s). Peak memory is a property of the whole run.

all families used in practice, including application-level and runtime choices; (ii) the governing memory and throughput equations written in one place so that stacking optimizations is not folklore; and (iii) a reproducible empirical slice of the taxonomy on Apple Silicon, where CPU and GPU share one DRAM pool and 7–9B models are the default product, not a research toy.

Scope. We survey inference-time methods: post-training compression, cache management, kernels, decoding algorithms, serving schedules, parallelism, runtimes, and prompt-side controls. We mention pruning, distillation, and architecture search only as they affect *serving* a finished model. Training-time algorithms (optimizer choice, 3D parallelism for pretraining) are out of scope except where the same primitive (tensor/pipeline/expert parallel) is reused at inference.

Organization. Section 2 reviews the inference pipeline and a unified memory budget. Section 3 states the taxonomy. Sections 4–13 treat each family. Section 14 reports Apple Silicon measurements. Section 15 gives a decision procedure. Section 16 lists open problems. Section 18 concludes.

2 Background: the inference pipeline

2.1 Prefill and decode

Let a request have prompt length T_p and generation length T_g, with $T = T_p + T_g$. Inference has two phases (Figure 1):

1. **Prefill.** The model consumes all T_p prompt tokens, writes the KV cache, and emits the first generated token. The user-visible metric is time-to-first-token (TTFT).
2. **Decode.** The model emits tokens $2, \ldots, T_g$ autoregressively. The user-visible metric is decode throughput (tokens/s). Each step typically attends over a cache of length $T_p + t$.

Confusing the two phases is the most common optimization error: a change that doubles tokens/s can leave TTFT untouched, and a Flash-style kernel can collapse TTFT on long prompts while barely moving streaming speed.

2.2 Memory budget

Peak resident memory is approximately

$$M_{\text{peak}} \approx M_{\text{weights}} + M_{\text{KV}} + M_{\text{act}} + M_{\text{overhead}}, \tag{1}$$

with static weights

$$M_{\text{weights}} \approx \frac{N \cdot b_w}{8} \tag{2}$$

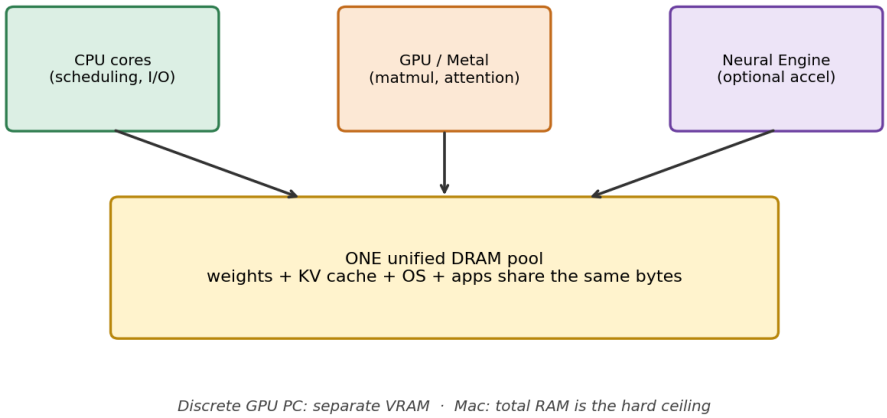

Figure 2: Unified memory on Apple Silicon: weights, KV cache, activations, and the operating system share one DRAM pool. Capacity planning is the first optimization.

for N parameters at b_w bits, and a KV cache

$$M_{\text{KV}} \approx 2\, L\, H_{\text{kv}}\, T\, D\, \frac{b_{\text{kv}}}{8} \tag{3}$$

for L layers, H_{kv} key/value heads, head dimension D, sequence length T, and b_{kv} bits per cached element [46, 3]. The factor of two is the pair (K, V). Grouped-query and multi-query attention reduce H_{kv} relative to query heads [51, 3].

For a Llama-class 8B model ($N \approx 8 \times 10^9$), $b_w = 16$ already costs ≈ 16 GB of weights. At $b_w = 4$ the same term is ≈ 4 GB. For $L = 32$, $H_{\text{kv}} = 8$, $D = 128$, $T = 640$, FP16 KV is ≈ 84 MB; 4-bit KV is ≈ 21 MB. Weights set the *floor*; KV sets the *slope* in T.

2.3 Roofline view of decode

When a decode step is memory-bandwidth bound [58],

$$\text{tok/s} \approx \frac{B_{\text{mem}}}{B_{\text{token}}}, \tag{4}$$

where B_{mem} is sustained DRAM (or HBM) bandwidth and B_{token} is bytes moved per generated token—dominated by reading the weights, plus a growing KV term. Reducing b_w therefore raises tokens/s *and* lowers M_{weights} on bandwidth-bound hardware. That is why 4-bit serving is not only a capacity trick.

2.4 Unified memory versus discrete GPUs

On a discrete GPU, weights live in HBM; the host CPU has a separate DRAM pool; PCIe (or NVLink) is an explicit copy path. On Apple Silicon, CPU, GPU, and Neural Engine share one unified DRAM pool [4, 5] (Figure 2). There is no host–device copy in the CUDA sense, but there is also *no private VRAM ceiling*: the OS, browser, KV cache, and weights compete for the same gigabytes. Local 8B inference on a 24 GB Mac is therefore a packing problem first and a kernel problem second.

2.5 Metrics

We use three primary metrics throughout: peak memory (GB), TTFT (ms or s), and decode throughput (tokens/s). Serving systems add goodput, time-to-last-token, and latency percentiles (p50/p99)

Table 1: Taxonomy of LLM inference optimizations. Primary metric is what a practitioner should look at first.

Family	Representative methods	Bottleneck	Primary metric
Weight / activation compression	FP16/BF16, INT8/4/2, GPTQ, AWQ, SmoothQuant, pruning, distillation	Capacity + decode bandwidth	Peak GB, tok/s, quality
KV cache & context	KV quant, GQA/MQA, PagedAttention, prefix cache, H2O, StreamingLLM	Capacity slope in T	Peak GB at long T
Attention & prefill	FlashAttention, chunked prefill, sparse/local attn, RoPE/YaRN	Prefill compute / IO	TTFT
Decode algorithms	Speculative decoding, Medusa, EAGLE, Jacobi, graphs	Sequential decode	tok/s, accept rate
Batching & scheduling	Continuous batching, SLOs, disaggregated PD	Multi-request utilization	Throughput, p99
Parallelism	Tensor, pipeline, expert, data parallel	Multi-device fit/latency	Tokens/s at scale
Architecture & size	Dense size ladder, MoE, long-context variants	Quality vs. fit	Pareto (quality, GB, tok/s)
Adapters	LoRA / QLoRA, multi-LoRA	Extra matmuls + RAM	Latency vs. specialization
Compilation & runtimes	`compile`, quantized GEMM, MLX, llama.cpp, offload	Kernel efficiency	tok/s at matched bits
Application-level	RAG sizing, prompt cache, constrained decoding, early exit	Tokens billed / prefilled	End-to-end latency

under load [2, 61]. Quality—perplexity, win rate, or task accuracy—is a constraint, not a free variable: an optimization that doubles speed at an unacceptable quality drop is a different model, not a faster one.

3 A bottleneck-first taxonomy

Table 1 organizes techniques by the bottleneck they attack. Families are complementary: weight bits do not replace KV paging; speculative decoding does not shrink the checkpoint.

A useful stacking order for *single-stream* local inference is: (1) choose a model that can fit; (2) quantize weights until there is OS headroom; (3) quantize or page KV if context grows; (4) tune prefill if TTFT hurts; (5) add speculation if decode still lags and RAM remains; (6) only then change runtime or hardware. Multi-tenant serving inverts the last steps: paging and continuous batching become mandatory before exotic decode algorithms. Figure 3 sketches the local funnel.

4 Weight and activation compression

4.1 Affine quantization

Affine quantization [28] maps a real weight x to a b-bit integer code

$$q = \mathrm{clip}\big(\mathrm{round}\big(\tfrac{x}{s} + z\big)\,,\, 0,\, 2^b - 1\big)\,, \qquad \hat{x} = s\,(q - z), \tag{5}$$

with scale s and zero-point z, typically shared by a group of G weights (e.g. 64 or 128). Group-wise scales keep dynamic range honest when a tensor has outliers. At inference, kernels dequantize on the fly or compute in low precision so a full FP16 copy is never materialized.

4.2 Post-training methods for LLMs

GPTQ [18] quantizes weights column-by-column, using a Hessian-aware update so remaining weights compensate for the rounding error. **AWQ** [37] identifies activation-salient channels and protects them (or rescales them) so aggressive 4-bit quantization does not destroy the directions the network actually uses. **LLM.int8()** [13] keeps outlier features in higher precision during 8-bit matmul. **SmoothQuant** [59] migrates quantization difficulty from activations into weights, enabling

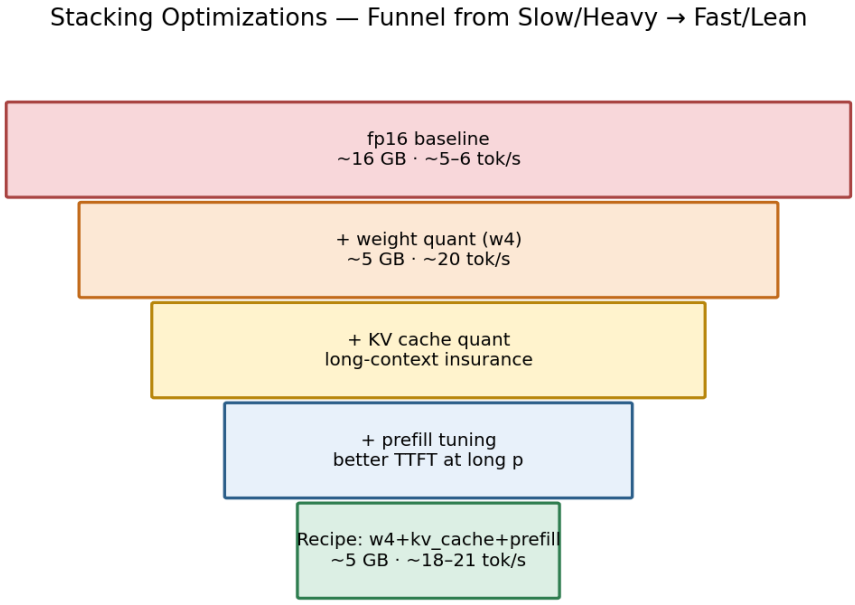

Figure 3: Local optimization funnel: weight bits shrink the floor, KV quantization insures growth in T, prefill tuning attacks TTFT, and the daily recipe is the combination that survives the RAM budget.

W8A8-style pipelines. Sparse-quantized hybrids such as SpQR [15] and SqueezeLLM [31] isolate a tiny high-precision outlier set.

In deployed stacks, practitioners often *load* a pre-quantized checkpoint (GPTQ/AWQ/GGUF/MLX-community) rather than running the calibration procedure at serve time. Bit width $b_w \in \{16, 8, 4, 2\}$ is then a first-class serving knob. Empirically, 4-bit is the default sweet spot on unified-memory laptops: 8-bit still spends too much DRAM, 2-bit frequently shows quality cliffs (Section 14).

4.3 Activation quantization, mixed precision, pruning, distillation

Activation quantization (W8A8, W4A8, ...) matters when GEMM is compute-bound or when activation RAM dominates, which is more common in prefill and large-batch serving than in single-stream decode. Mixed precision (AMP) keeps accumulators in FP32 while storing weights in FP16/BF16; it is the training/inference default, not an extra flag, in most GPU stacks.

Pruning—structured or unstructured—removes weights [23, 17, 39]. Unstructured sparsity needs specialized kernels to beat dense 4-bit GEMM; structured (channel/head) pruning is easier to serve but harder to keep accurate. Knowledge distillation [24, 47] produces a smaller student: it is a *different model*, not a runtime switch, but it is often the highest-leverage "optimization" if a 1–3B student meets the product bar.

5 KV cache and context memory

5.1 Why the cache exists

Scaled dot-product attention [56] is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right) V. \tag{6}$$

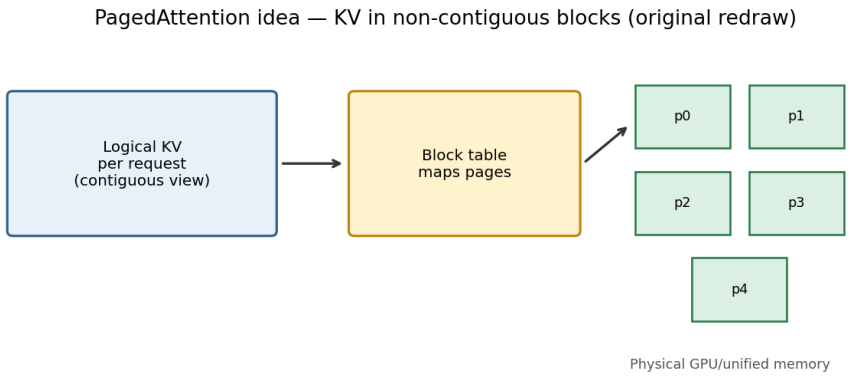

Figure 4: Paged KV (redraw after Kwon et al., 2023): multi-user serving stores the cache in blocks. Local single-stream inference is the single-tenant cousin of the same memory problem.

Recomputing K, V for the entire history at every decode step would repeat $O(T)$ work per token. Caching per-layer K and V makes decode $O(1)$ cache appends plus attention against the stored history. Equation (3) then grows linearly in T.

5.2 Fewer KV heads

Multi-query attention [51] shares one KV head across query heads. Grouped-query attention [3] interpolates, with $H_{\text{kv}} \ll H_q$. This is an *architectural* reduction of M_{KV} and of cache bandwidth; it is not a serving flag, but it explains why two 7B models can have very different long-context RAM slopes.

5.3 Quantizing the cache

KV quantization stores cached K, V at $b_{\text{kv}} < 16$ (commonly 8 or 4). It is independent of b_w: one can run FP16 weights with 4-bit KV, or 4-bit weights with FP16 KV. At short chat lengths the absolute savings are modest (tens of megabytes). At long generation or RAG context they become the difference between fitting and swapping. Quality impact is usually smaller than aggressive weight quantization because each decode step only reads the cache rather than using it as the model's entire knowledge.

5.4 Paging, prefix reuse, and eviction

PagedAttention [32] stores KV in non-contiguous blocks, analogous to virtual memory, so a serving engine can pack many sequences without reserving a giant contiguous buffer per request (Figure 4). **Prefix / prompt caching** reuses KV for a repeated system prompt or RAG template; the first request pays prefill, later requests skip it. **Eviction** methods drop tokens believed to be unimportant: H_2O keeps heavy hitters [62]; StreamingLLM keeps an attention sink plus a sliding window [60]. Sliding-window attention (e.g. Mistral-style) bounds KV by construction [29]. These techniques trade exact long-range attention for a bounded RAM envelope.

6 Attention kernels and prefill

Naive attention materializes $S \in \mathbb{R}^{n \times n}$ per head. At $n = 2048$ and FP16 that is already $\approx 8\,\text{MB}$ per head per layer, before the matmuls. FlashAttention [12] tiles Q, K, V through on-chip SRAM, uses an online softmax [41], and never writes the full score matrix to HBM. FlashAttention-2/3

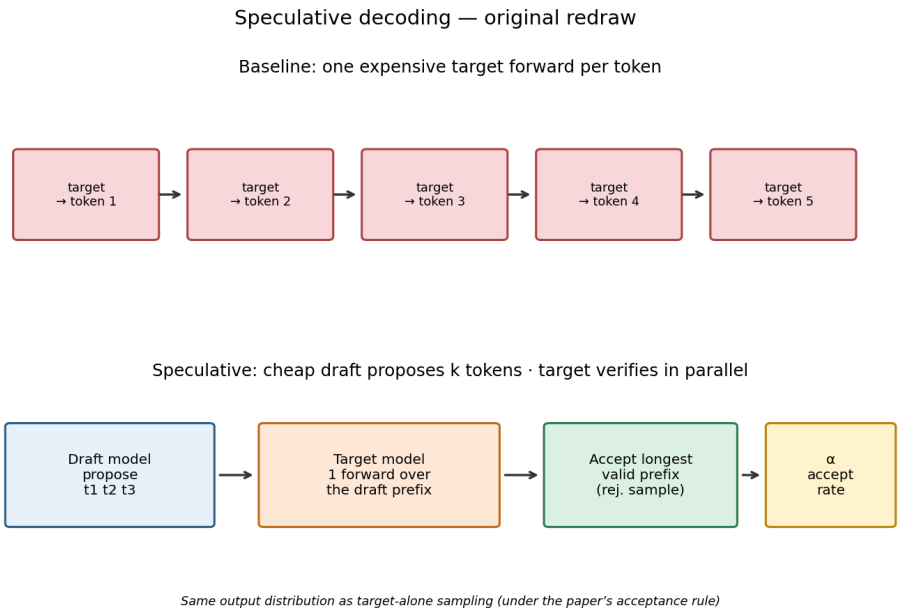

Figure 5: Speculative decoding (redraw after Leviathan et al. and Chen et al.): a draft proposes k tokens; the target verifies a prefix. Speedup tracks acceptance α, not merely the presence of a draft.

improve parallelism and low-precision paths [11, 50]. On Apple Silicon, MLX implements tiled Metal attention internally; user code rarely toggles a "Flash" flag.

Chunked prefill (also called prefill step size) processes the prompt in blocks of C tokens [1, 2]. Smaller C lowers peak activation memory and can improve mixing of prefill with decode in a server; larger C often lowers TTFT when the device is under-occupied. In local MLX measurements, $C = 2048$ versus $C = 512$ is the explicit knob corresponding to this family.

Sparse and local attention [10, 7] reduce the $O(n^2)$ term by restricting the pattern. State-space and linear-time models [22] change the architecture rather than the kernel. Rotary position embeddings [55] and YaRN [45] extend usable context without a full pretrain; they do not by themselves reduce M_{KV}.

7 Decode-time algorithms

7.1 Speculative decoding

Baseline decode costs $\approx T_g \cdot \tau_t$ where τ_t is the time for one target-model forward. Speculative decoding [34, 9] uses a cheap draft model to propose k tokens; the target verifies them in one parallel forward and accepts a prefix of length $m \leq k$ under a rejection-sampling rule that preserves the target distribution (Figure 5).

Let α be the per-token acceptance probability. Expected accepted tokens per round grow with α and k; wall-clock speedup requires $\tau_d \ll \tau_t$ and high α. If the draft is a poor match, one still pays $\tau_d + \tau_{\mathrm{verify}}$ and can *lose* throughput. Medusa [8] attaches extra heads to the target instead of a separate draft. EAGLE [36] drafts at the feature level. Lookahead and blockwise parallel decoding [54, 19] are related multi-token proposals. Jacobi / parallel decoding [48] iterates several future positions at once.

CUDA graphs (or Metal capture) replay a static decode graph, cutting launch overhead once shapes stabilize. They help small-batch, latency-sensitive serving more than large-batch throughput.

8 Batching, scheduling, and disaggregation

Local `generate()` loops are single-stream. Production serving is not.

Static batching waits to fill a batch of size B; it under-utilizes the GPU when arrivals are bursty and forces padding. **Continuous batching** (iteration-level scheduling), pioneered in Orca [61] and now standard in vLLM [32], mixes prefill of new requests with decode of in-flight ones at every step. The unit of scheduling is a model forward, not a whole request.

Chunked prefill in Sarathi-Serve [2] piggybacks decode tokens alongside partial prefills so a long prompt cannot stall interactive traffic. **Disaggregated prefill/decode** [44, 64] places the two phases on different pools, because prefill is compute-heavy and decode is KV-/bandwidth-heavy; goodput, not raw tokens/s, is the objective.

Request scheduling adds SLOs, preemption, and priority (chat versus batch jobs). These policies do not change the math of Equations (1)–(4); they change *who* holds the KV pages and *when* a prefill is admitted.

9 Parallelism

When one device cannot hold the model, inference reuses training sharding:

- **Tensor parallelism** [53, 42] splits GEMMs inside a layer. Latency can drop if interconnect is fast; it *rises* if all-reduce dominates a small decode step.
- **Pipeline parallelism** [27] assigns layer stages to devices. Bubble time is painful at batch 1.
- **Expert parallelism** for MoE [16, 33, 30] routes tokens to expert shards. Memory is proportional to *total* experts; compute per token is proportional to *active* experts.
- **Data parallelism** replicates weights and splits the batch; it is natural for large-batch throughput, less so for a single interactive user.

Consumer Macs almost never expose multi-GPU tensor parallel for LLMs; the relevant decision is "does this dense 70B fit in unified memory at 4-bit?" rather than "how many NVLink hops?" Datacenter serving inverts that.

10 Architecture and model-size choices

The highest-leverage "optimization" is sometimes a smaller model. Memory (2) and bandwidth-bound decode (4) both scale with N. A dense 0.5B 4-bit model and a dense 8B 4-bit model can differ by an order of magnitude in tokens/s on the same laptop (Section 14). Mixture-of-experts [30, 38] offers a different Pareto curve: more total parameters (quality/capacity) at a smaller active-FLOP budget. Long-context variants (32k–128k windows) do not change N but inflate T in (3) and prefill cost. Instruction-tuned versus base checkpoints should be held fixed in systems benchmarks; they are not speed knobs.

11 Adapters

LoRA [26] injects low-rank updates BA into frozen weights. QLoRA [14] fine-tunes those adapters on a quantized base. At inference, each adapter adds a small extra GEMM and a small extra RAM footprint. Multi-LoRA serving keeps many adapters resident and swaps per request; the systems problem is cache locality and batching across different B, A pairs, not just base-model paging. For a single local user, one LoRA is usually noise in the memory budget compared with b_w.

12 Compilation, kernels, and runtimes

Graph compilation (`torch.compile`, `mlx.compile`, XLA, TensorRT-LLM) fuses ops and specializes shapes. Quantized matmul kernels determine whether 4-bit is actually faster than FP16 or merely smaller: naive dequant-then-FP16-GEMM can lose the roofline win. CPU/disk offload [52]

runs models that do not fit, at a large bandwidth penalty. Memory-mapping weights from SSD reduces cold-start RAM spikes.

Local Mac runtimes illustrate that *the same bit width is not the same system*:

- **MLX / mlx-lm** [5, 6]: Metal-native arrays, Hugging Face `mlx-community` checkpoints, Python harnesses.
- **llama.cpp / GGUF** [21, 20]: C/C++ engine, Metal/CUDA/CPU backends, community GGUF quants (e.g. Q4_K_M).
- **Ollama** [43]: distribution and UX over llama.cpp-class engines.

Fair comparison requires matched prompts, generation length, and a comparable quant *tier* (FP16 vs. F16 GGUF; 4-bit MLX vs. Q4_K_M), not matched marketing names.

13 Application-level inference

Not every latency problem belongs inside the model.

Retrieval-augmented generation [35] can *increase* TTFT by stuffing thousands of tokens into T_p. The correct optimization is often "retrieve less" or "rerank then truncate," not a new kernel. Prefix caches (Section 5) absorb repeated templates. Semantic prompt caches reuse prior completions for identical or near-identical queries. Constrained / structured decoding [57, 63] masks logits toward JSON or grammars; it adds CPU work per token and can reduce sample diversity. Early-exit and confidence stopping [49] cut T_g when the model is already sure. Nucleus and temperature [25] are quality knobs with mild speed effects (via average T_g and cache traffic).

14 Empirical study: local inference on Apple Silicon

The taxonomy above is only useful if the local, measurable subset is actually measured. We summarize a public harness[2] that holds prompt length, generation length, and trial policy fixed, and varies one optimization family at a time and then in combination. Hardware: Mac M3 (24 GB unified memory) and Mac M5 Max. Default shape unless noted: $T_p = 512$, $T_g = 128$, median over measured trials after one warmup. Models are mlx-community checkpoints (Llama 3.1, Mistral, Qwen, Gemma, and related presets). Configurations are labeled `fp16`, `w8`, `w4`, `w2`, with optional `+kv_cache` (4-bit KV) and `+prefill` (`prefill_step_size` $= 2048$ vs. 512).

14.1 Weight quantization dominates the floor

On Mac M3, Llama 3.1 8B at FP16 uses **16.33 GB** peak and decodes at **5.6 tokens/s** with TTFT ≈ 2.7 s. The same model at 4-bit drops to ≈ 5.1 GB and rises to ≈ 20.5 tokens/s—the roofline prediction (4) in miniature: fewer bytes per weight, more tokens per second, and enough headroom for the OS. Across 14 M3-feasible presets, 4-bit is consistently the knee of the memory–speed Pareto (Figure 6). 2-bit saves additional RAM but is the first place quality complaints concentrate; we treat it as a last resort, not a default.

14.2 Stacking KV quantization and prefill

Table 2 reports the capstone comparison. Adding 4-bit KV and larger prefill chunks on top of 4-bit weights yields **5.06 GB** and **19.9 tokens/s** for Llama 3.1 8B on M3 ($\approx 3.55\times$ decode vs. FP16, $\approx 31\%$ of FP16 memory). Mistral 7B moves from 3.6 to 16.0 tokens/s and 14.8 to 4.6 GB. TTFT is not automatically improved by weight quant alone; long-prompt TTFT is a prefill-kernel and T_p problem (Section 6).

KV quantization's absolute gigabyte win is small at $T = 640$ (Equation (3)), as theory predicts. It becomes the right lever when T_g or T_p grows into the thousands, and when many sequences share a device (paging).

[2] Companion repository: LLM-Inference, MLX/mlx-lm sweeps with isolated subprocesses and versioned JSON.

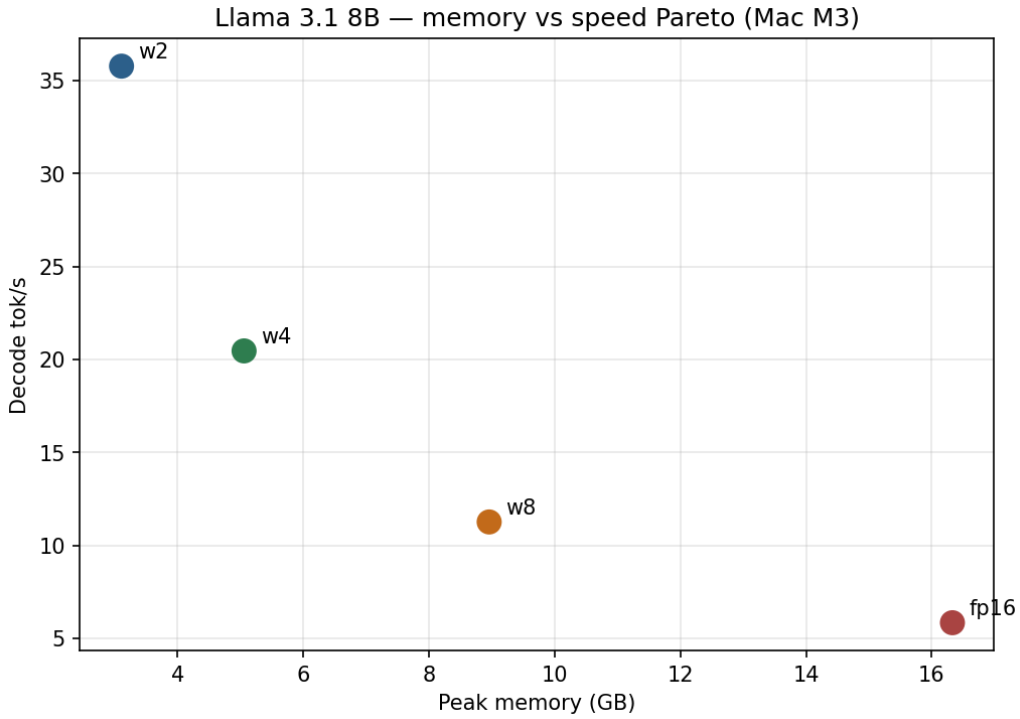

Figure 6: Memory–speed Pareto across weight bit-widths and models on Apple Silicon. Moving from FP16 to 4-bit is a joint win on both axes for bandwidth-bound decode.

Table 2: Full stack on Mac M3 (24 GB), $T_p = 512$, $T_g = 128$. Fully optimized = `w4+kv_cache+prefill`.

Model / recipe	Peak GB	TTFT (ms)	tok/s
Llama 3.1 8B, fp16	16.33	2689	5.6
Llama 3.1 8B, full stack	5.06	2746	19.9
Mistral 7B, fp16	14.8	—	3.6
Mistral 7B, full stack	4.6	—	16.0

14.3 Model-size ladder

Holding the recipe at 4-bit, M3 decode spans **238 tokens/s** at Qwen 2.5 0.5B (0.64 GB) down to **20.6 tokens/s** at Llama 3.1 8B (5.06 GB) and **15.4 tokens/s** at Gemma 2 9B (5.88 GB). On M5 Max, Llama 8B 4-bit reaches ≈ 112 tokens/s and Gemma 27B 4-bit remains interactive at ≈ 33 tokens/s. Hardware bandwidth multiplies the same software stack; it does not replace quantization on a 24 GB machine.

14.4 Speculative decoding is not unconditionally faster

With a small 4-bit draft, Qwen 2.5 7B on M3 goes from 15.9 to **28.3 tokens/s** ($1.78\times$) at acceptance $\alpha = 74.2\%$, adding only ≈ 0.3 GB. The same pair on M5 Max goes from 122 to 170.3 tokens/s at the same α. Llama 3.1 8B on M5 Max is a negative result: 113.5 $\rightarrow$ 109.7 tokens/s at $\alpha = 58.6\%$. Speculation should be reported with α and a baseline, never as a guaranteed multiplier.

14.5 Context, RAG, and prefix cache

Prefill TTFT grows steeply with T_p. A `rag_agent` workload with $T_p = 4096$ on Llama 3.1 8B 4-bit records TTFT ≈ 31.1 s on M3 versus ≈ 1.5 s on M5 Max. That is a product failure on the smaller chip even when decode tokens/s look acceptable. Prefix KV cache converts a repeated

Table 3: Symptom → first optimization. Later rows assume earlier ones are already sane.

Symptom	First lever
OOM / swap thrash	Smaller N, then $b_w \to 4$
Fits but slow stream	b_w, then faster DRAM class
Fine on chat, dies on RAG	Shorter T_p, KV quant, prefix cache, Flash/chunked prefill
Spinner before first token	Reduce T_p; raise prefill efficiency
Slow stream, RAM left over	Speculative decoding if α is high
Many concurrent users	Paged KV + continuous batching
Model larger than one device	TP/PP/EP or MoE routing
Need a specialist without a new base	LoRA at inference
Same bits, disappointing speed	Runtime / kernel / compile
Quality regression after 4-bit	W8 or better quant method; do not use W2 as default

system prompt from a prefill into a cache hit; it is the local analogue of a provider's prompt cache. Generation-length sweeps confirm Equation (3): decode tokens/s decay slowly as attention over the cache grows; peak memory tracks T.

14.6 Runtimes

MLX and llama.cpp implement overlapping quant tiers on the same Metal GPU. In matched-tier comparisons (FP16 vs. F16 GGUF; 4-bit vs. Q4_K_M), neither engine universally wins; kernel quality, thread pinning, and quant layout dominate. Ollama is a packaging layer, not a third algorithm family. The practical rule is: pick the runtime that exposes the lever you need (MLX for this paper's Python sweeps; llama.cpp for portable GGUF and CPU fallback), then freeze it while you sweep b_w and T.

15 A decision procedure

Table 3 maps the failure the user actually feels onto the first lever. The order is intentional: capacity before kernels, kernels before speculation, single-stream before multi-tenant.

Figure 7 shows the empirical payoff of following the local path on Llama 3.1 8B.

16 Open problems

Several gaps remain after this stack.

Quality under compression. Systems papers still under-report task-level quality at 4- and 2-bit, especially for tool use and long-context reasoning. A standard, cheap eval suite that travels with every quant checkpoint would change practice more than another 5% kernel win.

Unified-memory theory. Roofline models assume a GPU with private HBM. Unified DRAM, OS interference, and background apps make B_{mem} a random variable. We need measurement protocols that treat the laptop as a contended device, not an idle accelerator.

When speculation fails. Acceptance α is not a property of the algorithm alone; it is a property of the draft–target pair, temperature, and domain. Adaptive k, draft routing, and "disable speculation when α drops" should be first-class runtime features.

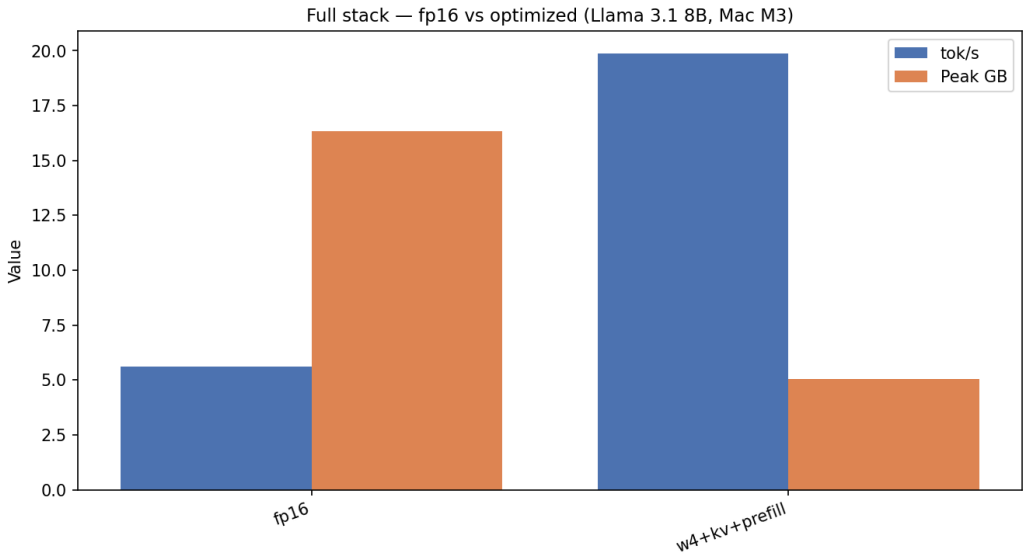

Figure 7: Mac M3, Llama 3.1 8B: FP16 versus the stacked 4-bit + KV + prefill recipe. Memory falls to $\approx 31\%$ of baseline; decode rises $\approx 3.55\times$.

Prefill as a first-class SLO. Industry dashboards still over-index on decode tokens/s. RAG and agents die on TTFT. Chunked prefill, prefix caches, and retrieve-less application design need the same experimental status as weight bits.

Cross-runtime equivalence. "4-bit" in MLX, GPTQ, AWQ, and GGUF Q4_K_M are not the same random variable. A quant-tier ontology with shared perplexity probes would make runtime bake-offs honest.

Energy and thermals. Tokens/s on a MacBook is eventually a skin-temperature problem. Joules per token, not only wall-clock, should appear in local inference reports.

17 Related surveys

Miao et al. [40] survey generative serving from algorithms to systems, with emphasis on datacenter engines. Zhou et al. [65] survey efficient LLM inference with a broad algorithm catalog. Our taxonomy is aligned with both and differs in three respects: we write the stacked memory budget (1)–(3) as the organizing math; we include application-level and local-runtime families that those surveys treat lightly; and we attach a unified-memory empirical study so that the local subset of the catalog is not only qualitative.

18 Conclusion

LLM inference optimization is not one trick. It is a packing problem (does the model fit?), a bandwidth problem (how many bytes per token?), a prefill problem (how long is the prompt?), a sequential-decode problem (can we verify several tokens at once?), and—at scale—a scheduling problem (who holds the KV pages?). The techniques in this survey attack those bottlenecks independently and can be stacked. On a 24 GB Mac, the stack that matters first is mundane and decisive: 4-bit weights, KV insurance for long T, and prefill hygiene, with speculative decoding as a conditional bonus when acceptance is high. The same catalog, read from the other end, is vLLM-style paging, continuous batching, and disaggregated prefill/decode. We hope the taxonomy, equations, and measurements make it obvious which end of the catalog to open for a given failure mode.

Acknowledgments and Disclosure of Funding

Benchmark numbers in Section 14 were collected with an open MLX harness on Apple Silicon. We thank the authors of the cited systems and the mlx-community and GGUF contributors who make comparable checkpoints publicly available. Update this acknowledgment with funding and hardware donors before camera-ready or arXiv v1 if applicable.

References

[1] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nagarajan Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. Sarathi: Efficient LLM inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nagarajan Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.

[3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

[4] Apple Inc. Apple announces Mac transition to Apple silicon. `https://www.apple.com/newsroom/2020/11/apple-announces-mac-transition-to-apple-silicon/`, 2020.

[5] Apple Inc. MLX: Efficient machine learning on Apple silicon. `https://github.com/ml-explore/mlx`, 2023.

[6] Apple Inc. mlx-lm: LLM utilities for MLX. `https://github.com/ml-explore/mlx-lm`, 2023.

[7] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[8] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

[9] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau Driessche, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[10] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[11] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

[12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[13] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, 2022.

[14] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized language models. *Advances in Neural Information Processing Systems*, 2023.

[15] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. *arXiv preprint arXiv:2306.03078*, 2023.

[16] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[17] Elias Frantar and Dan Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. 2023.

[18] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.

[19] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Accelerating large language model decoding with lookahead decoding. *arXiv preprint arXiv:2402.02057*, 2024.

[20] Georgi Gerganov. GGUF: GPT-generated unified format. `https://github.com/ggerganov/ggml/blob/master/docs/gguf.md`, 2023.

[21] Georgi Gerganov and contributors. llama.cpp. `https://github.com/ggml-org/llama.cpp`, 2023.

[22] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[23] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *International Conference on Learning Representations*, 2016.

[24] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[25] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. 2020.

[26] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations*, 2022.

[27] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, et al. GPipe: Efficient training of giant neural networks using pipeline parallelism. *arXiv preprint arXiv:1811.06965*, 2018.

[28] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018.

[29] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.

[30] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

[31] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. SqueezeLLM: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629*, 2023.

[32] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

[33] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. *International Conference on Learning Representations*, 2021.

[34] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[35] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 2020.

[36] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.

[37] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, 2024.

[38] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chengzhi Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[39] Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-Pruner: On the structural pruning of large language models. *Advances in Neural Information Processing Systems*, 2023.

[40] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. Towards efficient generative large language model serving: A survey from algorithms to systems. *arXiv preprint arXiv:2312.15234*, 2023.

[41] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.

[42] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. Efficient large-scale language model training on GPU clusters using Megatron-LM. 2021.

[43] Ollama. Ollama. `https://ollama.com`, 2023.

[44] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*, 2024.

[45] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023.

[46] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, James Devlin, Jacob Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*, 2023.

[47] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[48] Andrea Santilli, Alessandro Severino, Emanuele Postolache, Valentino Maiorca, Michele Mancusi, Riccardo Marin, and Emanuele Rodolà. Accelerating transformer inference for translation via parallel decoding. *arXiv preprint arXiv:2305.10427*, 2023.

[49] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. *Advances in Neural Information Processing Systems*, 2022.

[50] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *arXiv preprint arXiv:2407.08608*, 2024.

[51] Noam Shazeer. Fast transformer decoding: One write-head is all you need. 2019.

[52] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[53] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[54] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 2018.

[55] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[56] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

[57] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

[58] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[59] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[60] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *International Conference on Learning Representations*, 2024.

[61] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, 2022.

[62] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 2023.

[63] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, et al. SGLang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems*, 2024.

[64] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Huang, Yibo Zhu, Xin Zhang, Xin Wang, Hao Zhang, Haibo Li, et al. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.

[65] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, et al. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294*, 2024.

Table 4: Technique checklist (survey coverage).

Technique	Section	Typical status
FP16 / BF16 baseline	4	measured
INT8 / 4-bit / 2-bit weights, GPTQ, AWQ	4	measured
Activation quantization, SmoothQuant	4	systems
Pruning, distillation	4	reference
KV cache quantization	5	measured
GQA / MQA	5	architecture
PagedAttention	5, 8	systems
Prefix / prompt KV cache	5	measured
H2O, StreamingLLM, sliding window	5	systems / architecture
FlashAttention / tiled attention	6	measured (proxy)
Prefill chunking	6	measured
Sparse attention, RoPE, YaRN	6	architecture
Speculative decoding, Medusa, EAGLE	7	measured / systems
CUDA graphs / Metal capture	7	systems
Continuous and static batching	8	systems
Disaggregated prefill/decode	8	systems
Tensor / pipeline / expert parallel	9	systems
Model-size ladder, MoE	10	measured / reference
LoRA / multi-LoRA	11	systems
Graph compile, offload, mmap	12	systems
MLX vs. llama.cpp	12	measured
RAG sizing, constrained decoding, early exit	13	reference

A Catalog checklist

Table 4 is a compact checklist corresponding to Section 3. Status tags refer to the companion Apple Silicon harness: *measured* means a sweep exists; *systems* means the method is standard in datacenter stacks; *architecture* means it is baked into the checkpoint.

B Reproducing the empirical slice

On Apple Silicon, with Python 3.10+ and Hugging Face access:

```
git clone <repo> LLM-Inference && cd LLM-Inference
./scripts/setup_env.sh && source .venv/bin/activate
./scripts/hf_login.sh
./scripts/run_article.sh 1 "Mac M3"   # weight quantization
./scripts/run_article.sh 5 "Mac M3"   # full stack
./scripts/run_article.sh 6 "Mac M3"   # speculative decoding
```

Each run writes schema-versioned JSON (median TTFT, tokens/s, peak GB). Tables in Section 14 are medians from that schema, not hand-edited spreadsheets. Large-model rows require a 64 GB+ machine and `-include-large`.

C arXiv and NeurIPS usage notes

This source uses the official NeurIPS 2025 style with the `preprint` option, which is the recommended path for arXiv. It is *not* an anonymous conference submission: authors are visible, line numbers are off, and the NeurIPS paper checklist is omitted. To submit to a NeurIPS *workshop* (single-blind), switch the package line to `sglblindworkshop` and set `\workshoptitle{...}`. The main conference track is a poor fit for a survey unless the empirical protocol is framed as a datasets-and-benchmarks contribution and the page limit is respected (typically 9 pages of main text plus references). For arXiv, no page limit applies; we recommend keeping the main body focused and moving checklists to the appendix, as done here.